

PROIECT MASURATORI ELECTRONICE SI SENZORI

Suport de camera cu urmarire automata

Masuratori electronice si senzori

Bogdan Vlad

Grupa 30229

Facultatea de automatica si calculatoare

Cuprins

• Lista de abrevieri și simboluri	4
• Introducere	5
• 2.1 Justificarea abordarii temei	5
• 2.2 Importanta și actualitatea temei	5
• 2.3 Originalitatea și aplicabilitatea lucrării	6
• Descrierea realizării practice	7
• 3.1 Principiul functional	7
• 3.2 Schema electrica și componente	8
• 3.3 Probleme nerezolvate și motivarea lor	13
• 3.4 Observatii privind rezultatele obținute	13
• Concluzii	14
• Bibliografie	15

Lista de abrevieri si simboluri

- **AI** – Inteligenta Artificiala (Artificial Intelligence)
- **OpenCV** – Open Source Computer Vision Library
- **OX** – Axa orizontala de coordonate
- **OY** – Axa verticala de coordonate
- **OZ** – Axa de adancime de coordonate
- **UART/Serial** – Protocol de comunicatie intre PC si Microcontroler
- **RPM** – Rotatii pe minut

INTRODUCERE

2.1 Justificarea abordării temei

Necesitatea unui sistem de monitorizare foto-video de buget redus pentru scopuri simple diverse folosind AI pentru procesarea imaginii si un brat alcatuit din doua motoare pentru o permanenta incadrare a fetei oferind utilizatorului flexibilitate in ceea ce priveste miscarea

2.2 Importanța și actualitatea temei

În contextul actual, sistemele de urmarire facială sunt esentiale în domenii precum videoconferințele inteligente, supravegherea autonoma și robotica socială. Automatizarea mișcării camerei permite mentinerea subiectului în centrul atentiei fara interventie umana manuala.

2.3 Originalitatea si aplicabilitatea lucrarii

Originalitatea proiectului consta în utilizarea a două tipuri diferite de motoare pentru controlul axelor: un step motor (28BYJ-48) pentru precizie pe orizontal-verticala si un servomotor (SG90) pentru viteza pe axa oz. Sistemul poate fi aplicat ca suport de camera "smart" pentru creatori de continut sau in sisteme de control al accesului.

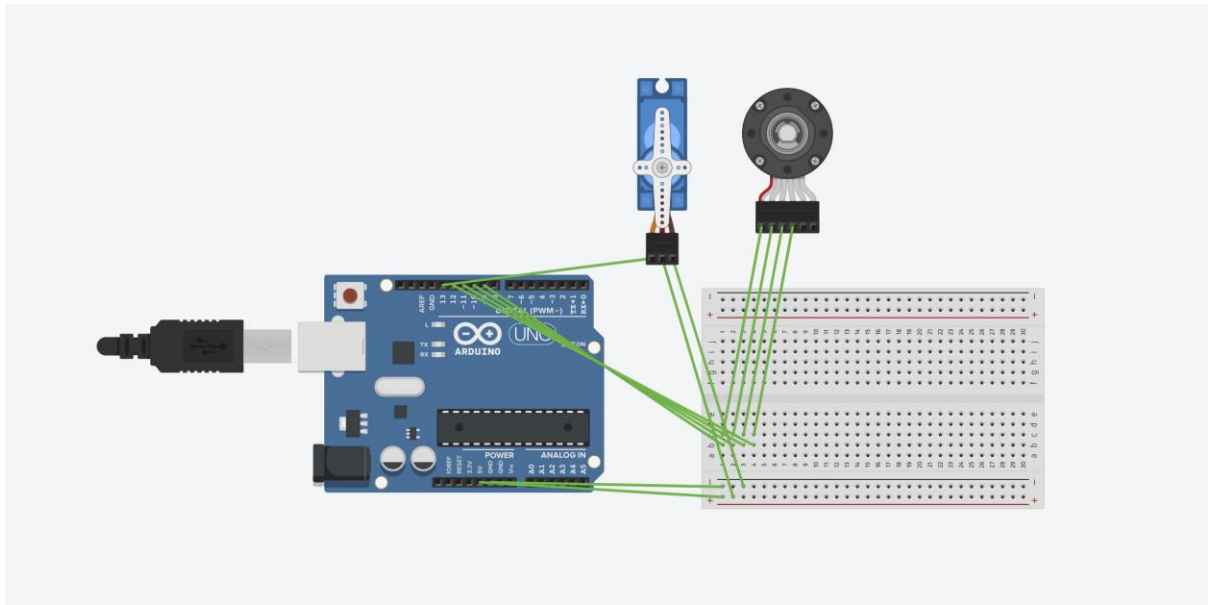
Descrierea realizarii practice

3.1 Principiul functional

Sistemul utilizeaza o bucla de control de tip "feedback" intre un algoritm de detectie facială si organele de executie mecanice.

1. **Algoritmul de detectie:** Scriptul Python utilizeaza biblioteca OpenCV si clasificatorul Haar Cascade pentru a identifica coordonatele (x, y, z) ale centrului feței în fluxul video.
2. **Calculul erorii:** Se determină deviatia fetei fata de centrul imaginii (setpoint).
3. **Transmisia datelor:** Comenzile de corectie sunt trimise prin protocol Serial (USB) sub forma de caractere ('L' - Left, 'R' - Right, 'U' - Up, 'D' - Down).
4. **Executia:** Arduino receptionează caracterele și comandă driverul ULN2003 (pentru Stepper) sau semnalul PWM (pentru Servo) pentru a orienta camera spre fata.

3.2 Schema electrică



Componente utilizate:

- Placa de dezvoltare tip Arduino Uno.
- Motor pas cu pas 28BYJ-48 cu driver ULN2003 (Pinii 9, 10, 11, 12).
- Servomotor SG90 (Pin 13).
- Cameră web USB.
- Sursa de alimentare (USB).
- Breadboard

Materiale confecție:

- Disc carton: ales pentru forma ce permite orientarea bratului pe axa ox respectiv oy fara a ocupa spatiu si a permite centrarea camerei nu a bratului complet si pentru greutatea redusa ce permite o mai usoara miscare a step motorului ce

misca intregul brat,oferind si o stabilitate mai buna(raport rezistanta-greutate bun)

- Profil aluminiu: ce permite o mai buna sustinere decat cartonul,materialul permitand atat rezistenta sporita cat si maleabilitate iar forma acestuia ofera practicabilitate in ceea ce priveste confectionarea si alterarea profilului

Observatii:

1.Bratul este orientat la aproximativ 45 de grade ,detaliu ce permite pastrarea centrului de greutate al camerei cat mai aproape de centrul step-motorului,oferind suport mai bun la miscarile orizontale astfel usurandu-i si treaba.

2.Programul scris in Python ofera simplitate in ceea ce priveste intelegerea instructiunilor cat si o mai usoara “comunicare” cu placa Arduino

```
import cv2
import serial
import time

PORT_SERIAL = 'COM8' # Am setat portul 8
BAUD_RATE = 9600

try:
    # Inițializare conexiune
    arduino = serial.Serial(PORT_SERIAL, BAUD_RATE, timeout=1)
    time.sleep(2) # așteptăm ca Arduino să se reseteze după conectare
    print(f"Conectat cu succes la {PORT_SERIAL}")
except Exception as e:
```

```

print(f"EROARE: Nu m-am putut conecta la {PORT_SERIAL}.")
print("Asigură-te că Serial Monitor din Arduino IDE este ÎNCHIS.")
print(f"Detalii eroare: {e}")
exit()

# Încărcare model detecție facială
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')

# Pornire cameră web
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Eroare: Nu am putut accesa camera web.")
    exit()

print("Programul a pornit. Apasă 'q' pentru a închide.")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Oglindim imaginea pentru ca mișcarea să fie naturală (stânga mea = stânga
    # camerei)
    frame = cv2.flip(frame, 1)

    # Transformam imaginea în alb-negru pentru o mai bună citire a camerei
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detectăm fețele
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    img_h, img_w = frame.shape[:2]
    center_x, center_y = img_w // 2, img_h // 2

    # Desenăm centrul ecranului
    cv2.circle(frame, (center_x, center_y), 5, (0, 255, 0), -1)

    for (x, y, w, h) in faces:
        # Calculăm centrul feței detectate
        face_center_x = x + w // 2
        face_center_y = y + h // 2

        # Desenăm dreptunghiul în jurul feței
        cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)
        cv2.circle(frame, (face_center_x, face_center_y), 5, (255, 0, 0), -1)

    # Toleranță (pixeli)

```

```

tol = 60

# Logica de trimitere comenzi
try:
    # Axa OY - Servo
    if face_center_y < center_y - tol:
        arduino.write(b'U') # Up
    elif face_center_y > center_y + tol:
        arduino.write(b'D') # Down

    # Axa OX - Stepper
    if face_center_x < center_x - tol:
        arduino.write(b'L') # Left
    elif face_center_x > center_x + tol:
        arduino.write(b'R') # Right

except Exception as e:
    print(f"Eroare la trimiterea datelor: {e}")
    break

# Afișăm fereastra
cv2.imshow('Tracking Facial - Arduino COM8', frame)

# Ieșire din program la apăsarea tastei 'q'
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Curățare resurse
cap.release()
cv2.destroyAllWindows()
if 'arduino' in locals():
    arduino.close()
    print("Conexiune serială închisă.")

```

Codul din Python cu comentarii pentru a intelege fiecare pas

3.Placa Arduino ofera o mai usoara transmitere a semnalelor catre motoare

```

#include <Stepper.h>
#include <Servo.h>

```

```

const int stepsPerRevolution = 2048;
Stepper myStepper(stepsPerRevolution, 9, 11, 10, 12);
Servo myServo;

int servoPos = 90;

void setup() {
  Serial.begin(9600);
  myStepper.setSpeed(12);
  myServo.attach(13);
  myServo.write(servoPos);
}

void loop() {
  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == 'U') {
      if (servoPos > 10) servoPos -= 1;
      myServo.write(servoPos);
    }
    else if (command == 'D') {
      if (servoPos < 170) servoPos += 1;
      myServo.write(servoPos);
    }

    if (command == 'L') {
      myStepper.step(10);
    }
    else if (command == 'R') {
      myStepper.step(-10);
    }
  }
}

```

3.3 Probleme nerezolvate si motivarea lor

Vibratia bratului-motive diverse de la motoarele ce nu ofera o miscare perfect continua pana la materialele folosite mai precis imbinatiile bratului unde am folosit suruburi in mare parte ce nu potrivesc exact orificiului din profil

Mici erori in miscarea bratului-motive precum rezolutia redusa a camerei si sensibilitatea redusa la lumina, codul Python ce nu asigura precizie 100%

Viteza de raspuns intarziata-conectiunea camera-python-arduino-motor intarzie o mai buna viteza

3.4 Observatii privind rezultatele obtinute

Sistemul este complet funcțional. În urma testelor, s-a constatat ca o tolerantă de 60 de pixeli în algoritmul Python previne oscilațiile inutile ale motoarelor atunci când fata este relativ centrată. Timpul de răspuns total este de aproximativ 100ms.

CONCLUZII

Proiectul reprezintă o soluție de tip **Low-Cost**, demonstrând ca un sistem de urmărire poate fi realizat cu un buget minim. Din punct de vedere economic, utilizarea componentelor standard permite repararea și scalarea facilă a dispozitivului.

BIBLIOGRAFIE

- Arduino Reference, *Stepper Library*, [Online] la: www.arduino.cc/reference/en/libraries/stepper/
- OpenCV Documentation, *Face Detection using Haar Cascades*, [Online] la: docs.opencv.org
- Pierzchalski, M., *Controlul motoarelor cu Arduino*, Editura Tehnică, 2022.
- PySerial Project, *Serial Communication Documentation*, 2024.

